1 Task 1: Stride Pattern Prefetcher (SPP)

1.1 Design overview

SPP is an L1D prefetcher that works per instruction pointer (IP) and never crosses a page boundary. It has three parts.

- **IP table** (64 entries, direct-mapped). Each entry stores the IP tag, the last accessed cache line (12 bits), the last stride, a 2-bit uniform-stride confidence counter and an 8-bit stride-history signature. The index is `(ip ^ (ip>>6)) & 63` and the tag is `(ip>>6) & 0xFF`.
- **Pattern table** (256 entries, indexed by the signature). Each entry holds the predicted next stride and a 2-bit confidence. The signature is updated as `sig' = ((sig << 3) ^ (stride & 0x7F)) & 0xFF`, so it summarises the last few strides of that IP and maps to the stride expected next. A repeating sequence such as $(+1, +2, +2, +2, +3)^n$ therefore trains a chain of table entries.
- **Accuracy checker**. Two small counters (prefetches issued and useful prefetches) are evaluated every 128 prefetches and set the prefetch degree.

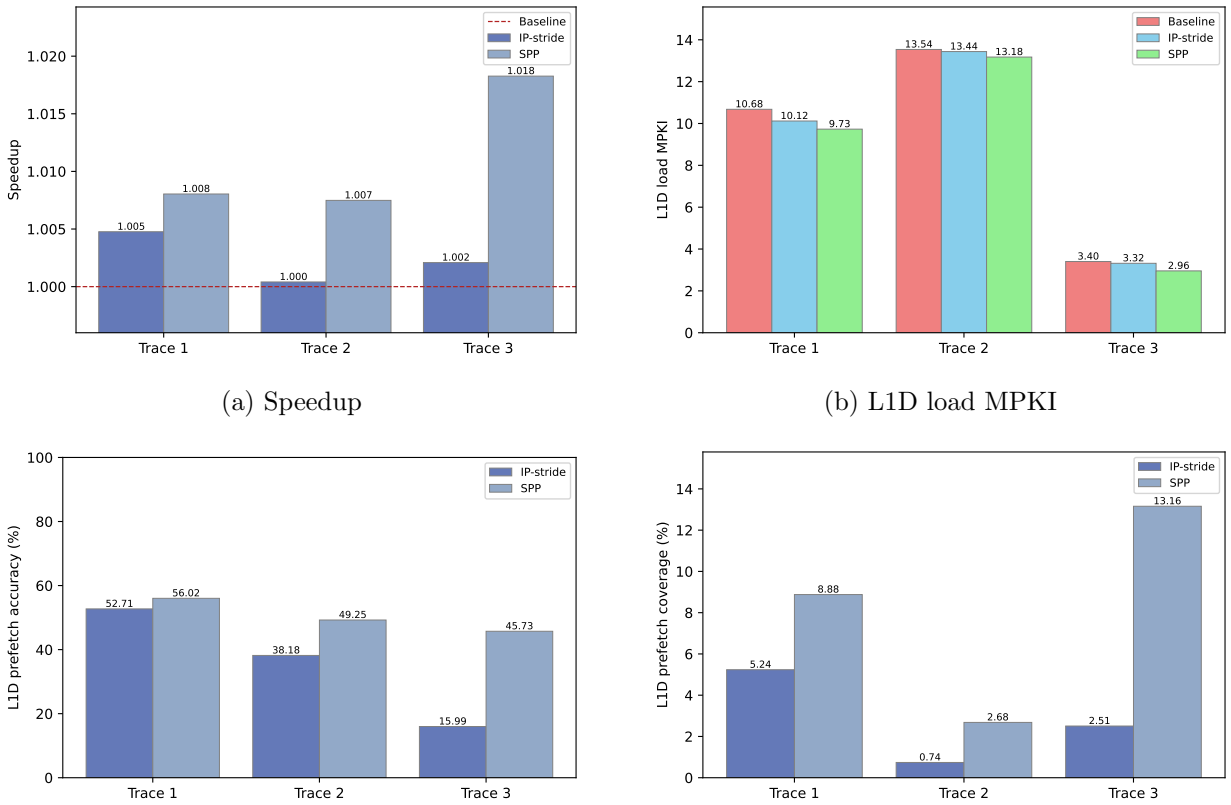
Uniform first, pattern as fall-back. If the uniform confidence of an IP is saturated (value 3), SPP prefetches `addr + k*stride` for $k = 1..$degree. When a stride differs from the previous one the counter is halved, so a broken uniform pattern drops out of the uniform path quickly, and the IP then uses the pattern table. In that mode SPP reads the predicted stride for the current signature, issues the prefetch, advances a speculative signature and repeats up to the current degree. It stops when the pattern-table confidence is below 2, the predicted stride is 0, or the next line would leave the page. Only demand accesses train the tables, and strides larger than 63 lines only resynchronise the IP entry. Prefetches are filled into L1D; if the L1D MSHR is more than half full the prefetch is dropped.

1.2 Results

Speedup is measured against the no-prefetching baseline of the same trace. L1D load MPKI and accuracy are taken from the ChampSim statistics (accuracy is useful prefetches divided by prefetches issued to the lower level), and coverage is the reduction in L1D load misses relative to the baseline.

Trace	Config	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace 1	Baseline	0.232345	1.0000	10.678	–	–
Trace 1	IP-stride	0.233452	1.0048	10.119	52.71	5.24
Trace 1	SPP	0.234213	1.0080	9.730	56.02	8.88
Trace 2	Baseline	0.251158	1.0000	13.538	–	–
Trace 2	IP-stride	0.251260	1.0004	13.438	38.18	0.74
Trace 2	SPP	0.253038	1.0075	13.175	49.25	2.68
Trace 3	Baseline	0.382065	1.0000	3.403	–	–
Trace 3	IP-stride	0.382862	1.0021	3.318	15.99	2.51
Trace 3	SPP	0.389045	1.0183	2.956	45.73	13.16

Table 1: IPC, speedup, L1D load MPKI, prefetch accuracy and coverage for the baseline, IP-stride and SPP.



(a) Speedup

(b) L1D load MPKI

(c) L1D prefetch accuracy

(d) L1D prefetch coverage

Figure 1: SPP compared with the IP-stride prefetcher and the baseline on the three traces.

Discussion. SPP gives a higher speedup than IP-stride on all three traces: 1.0080/1.0075/1.0183 against 1.0048/1.0004/1.0021, a geometric mean of 1.0113 against 1.0024. It also has higher coverage on every trace (8.88%, 2.68%, 13.16% against 5.24%, 0.74%, 2.51%) and higher accuracy. The pattern-table path is the reason. In all three traces most triggers come from non-uniform patterns: 330,222 pattern triggers against 113,216 uniform ones in Trace 1, 185,947 against 43,377 in Trace 2, and 337,323 against 47,749 in Trace 3. A constant-stride prefetcher cannot cover these accesses. Trace 3 benefits most: IP-stride has an accuracy of only 15.99% there, while SPP reaches 45.73% and a 13.16% coverage. The absolute gains are small because the traces are memory bound (baseline IPC between 0.23 and 0.38) and a large share of their load misses follow no stride pattern, so an in-page L1D stride prefetcher can cover only part of the miss stream. The ceiling is set by the traces and not by a single parameter, as the parameter study in Section 1.6 shows.

1.3 Storage overhead

Structure	Field	Bits/entry	Entries	Total bits
IP table	valid	1		
	tag	8		
	last cache line (12-bit)	12		
	last stride (signed)	7		
	uniform confidence	2		
	stride signature	8		
	entry total	**38**	64	**2432**
Pattern table	predicted stride (signed)	7		
	confidence	2		
	entry total	**9**	256	**2304**
Accuracy checker	issued counter (window 128)	8	1	8
	useful counter	8	1	8
	degree register	3	1	3
	total			**19**
Total				**4755 bits (594.4 B)**

Table 2: Bit-level storage of SPP. The pattern table is indexed directly by the 8-bit signature, so it needs no tag.

The issued counter counts prefetches sent to the lower level (an L1D miss) and the useful counter counts demand hits on prefetched L1D lines. Both are cleared at the end of every window of 128 issued prefetches, so an 8-bit counter is enough (the useful counter saturates).

1.4 Accuracy threshold for throttling

The accuracy of the last window (useful/issued) selects the degree: $\geq 75\% \rightarrow 6$, $\geq 50\% \rightarrow 4$, $\geq 25\% \rightarrow 2$, otherwise 1. The bands are evenly spaced quartiles. Below 25% fewer than one prefetch in four is useful, so extra prefetches mostly pollute L1D and use up bandwidth. Above 75% nearly every prefetch is used, so running further ahead is safe. The 50% boundary marks the point where useful prefetches outnumber useless ones. We compared this choice with a stricter set (90/70/40%) on Trace 1 (Section 1.6). The stricter thresholds lowered the degree more often, giving a coverage of 8.03% instead of 8.88% with no gain in IPC, so we kept 75/50/25%. The per-window accuracy varies widely between phases and traces (20.9% at the end of Trace 2, 28.1% for Trace 1 and 85.4% for Trace 3), and the degree histogram in Table 3 shows that all four degree levels are used, so every band of the throttle is exercised.

An earlier version counted every accepted prefetch call as "issued". That included requests that hit in L1D or merged in the prefetch queue and never reach the lower level, so the measured accuracy was far lower than the true accuracy and the degree stayed at 1 (Trace 1: 609,179 of 687,901 accesses at degree 1). Counting only prefetches that are sent to the lower level fixed this.

Trace	Windows	Last-window acc.	Final degree	d1	d2	d4	d6
Trace 1	920	28.1%	2	23,455	310,364	246,089	108,120
Trace 2	464	20.9%	1	124,257	72,756	74,853	131,208
Trace 3	601	85.4%	6	214,524	132,339	71,774	137,795

Table 3: Accuracy-checker behaviour: windows evaluated, accuracy of the last window, final degree, and the number of trained accesses spent at each degree.

1.5 Choice of prefetch degrees

- **Degree 6 ($\geq$ 75%).** Prefetches are rarely wasted, so SPP can run further ahead of the demand stream. This increases coverage and gives late prefetches more time to arrive.
- **Degree 4 ($\geq$ 50%).** Half of the prefetches pay off, so SPP keeps a moderate lookahead and limits pollution.
- **Degree 2 ($\geq$ 25%).** Most prefetches are useless, so SPP keeps only a short lookahead while some coverage remains.
- **Degree 1 (below 25%).** The prefetcher stays active so the tables keep learning and accuracy can recover when the access pattern changes, but pollution is minimal. Degrees 3 and 5 are not used, because the coarse steps make the throttle react clearly to a change of accuracy.

The chain depth is also limited by the pattern-table confidence and the page boundary, so the effective degree on a pattern-driven IP can be lower than the selected degree.

1.6 Parameter study on Trace 1

Two choices were tested on Trace 1 only: the accuracy thresholds, and whether prefetches that find the L1D MSHR busy are pushed to L2 or dropped. The IPC differences between the three variants are below 0.005%, so IPC does not separate them; coverage and accuracy do.

Variant	IPC	Speedup	MPKI	Cov. (%)	Acc. (%)
Initial (issued counter incl. L1 hits, window 256)	0.234072	1.0074	9.954	6.78	57.14
75/50/25, busy MSHR pushed to L2	0.234219	1.0081	9.736	8.82	54.52
90/70/40, busy MSHR pushed to L2	0.234208	1.0080	9.821	8.03	56.51
75/50/25, busy MSHR dropped **(final)**	0.234213	1.0080	9.730	8.88	56.02

Table 4: Design variants on Trace 1.

We chose 75/50/25 with dropped prefetches. It has the best coverage and lower load MPKI than the other threshold set, with essentially the same IPC, and it removes the path to L2.

L2C Instruction-Aware RRIP: Task 2a and 2b Report

ChampSim, 1 core, 25M warmup + 25M simulated instructions, 3 traces. Only the L2C replacement policy changes; LLC and all other levels use LRU.
Baseline = LRU at L2C. **Task 2a** = static I-RRIP.
Task 2b = dynamic I-RRIP (leader stride 32, epoch 4096).

Speedup = $\text{IPC}_{\text{policy}}/\text{IPC}_{\text{baseline}}$. Load MPKI values come from the `L2C LOAD`, `L2C INSTRUCTION LOAD` and `L2C DATA LOAD` MPKI lines of each output file. Averages of speedup are geometric means.

1 Task 2a: Static I-RRIP

1.1 Rationale for the insertion, promotion and eviction decisions

The policy (`irrip.l2c_repl`) uses a 2-bit RRPV (maximum 3) and one extra bit per line recording whether the resident line is an instruction line. All three decisions are modified, each in a small way.

Decision	SRRIP-style behaviour	I-RRIP (Task 2a)
Insertion	Every demand fill at RRPV 2	Demand instruction fill: **1**. Prefetched instruction fill: 2. Demand data fill: 2. Prefetched data or writeback fill: **3**.
Promotion	Hit resets RRPV to 0	Hit lowers RRPV by **one step** (floor 0), for every line kind.
Eviction	First line at RRPV 3	Still needs RRPV 3. If an instruction and a data line both sit at 3, evict the **data line**, except 1 time in 4 (global counter) when the instruction line goes.

Table 1: Policy parameters of Task 2a.

1. Why the problem exists. Trace2 has an L1I MPKI of 40.0 and a baseline L2 instruction load MPKI of 18.9, more than twice its L2 data load MPKI (8.7). Under LRU, instruction, data and translation lines are treated alike, so streaming data fills evict hot code. In trace2 the baseline recorded 139,041 data-evicts-instruction events.

2. Insertion: instruction lines start one step closer to reuse. A demand instruction fill enters at RRPV 1 against 2 for demand data, so it needs one more round of aging before it becomes a victim. Instruction fetches have strong temporal and spatial locality and an instruction miss stalls the front end, so a small head start is cheap insurance. Prefetched data and writeback fills enter at 3 (immediately evictable) because they are the least likely to be re-read soon. The gaps are deliberately small, one RRPV step: a large gap would become hard protection and starve data.

3. Promotion: gradual, not jump-to-zero. A hit moves the line down by one step instead of straight to 0. One hit is weak evidence of reuse, so a line has to earn full protection by repeated hits. A single stray touch cannot pin a line. It applies equally to instruction and data lines, which keeps the policy simple. `PROMOTE_ON_PF_WB_HIT` is 1, so prefetch and writeback hits also promote.

4. Eviction: RRPV decides, instruction bias only breaks ties. The victim must be at RRPV 3, exactly as in RRIP. The bias applies only when instruction and data lines are both at 3, i.e. both predicted dead. Data is then evicted by default, but one time in four (`INSTR_EVICT_PERIOD = 4`) the instruction line is evicted. Instruction priority therefore never becomes permanent: dead code is eventually cleared and a data-hot phase is never fully blocked. The counter is a global deterministic counter rather than `rand()`, so runs are reproducible.

5. Why this is the best static choice here.

- Largest gain where it matters: +1.97% IPC on trace2, with instruction MPKI down 23.9%. Instruction-evicts-instruction events fall 35.3%.
- Bounded cost: data load MPKI rises 1.2%–3.4% as data lines lose some capacity. Data-evicts-data events rise 15.1% on trace2.
- Trace1 gains only +0.08%: it is limited by branch accuracy (91.2%) and a 14.7% BTB miss rate, so the saved L2 instruction misses were not on the critical path. Trace3 has instruction MPKI of 1.1, so nothing is gained and nothing is lost (−0.006%).
- Cheap: one bit per line plus one global counter.
- The tie-break period is tunable. A larger period protects code harder and costs data more.

Limitation: the data-MPKI increase is unconditional in the static design, which motivates Task 2b.

Check against the assignment goal (reduce L2 load instruction MPKI): achieved on all three traces: 10.37 to 10.09 (trace1), 18.89 to 14.38 (trace2), 1.12 to 1.08 (trace3), average 10.13 to 8.52.

1.2 Results

Trace	Policy	IPC	Speedup	Load MPKI	Instr MPKI	Data MPKI
trace1	Baseline (LRU)	0.232246	1.0000	17.7027	10.3682	7.3345
trace1	I-RRIP	0.232440	1.0008	17.6615	10.0929	7.5686
trace2	Baseline (LRU)	0.251304	1.0000	27.5596	18.8918	8.6677
trace2	I-RRIP	0.256243	1.0197	23.3356	14.3775	8.9580
trace3	Baseline (LRU)	0.381855	1.0000	3.9603	1.1237	2.8366
trace3	I-RRIP	0.381833	0.9999	3.9505	1.0805	2.8700
Average	Baseline (LRU)	0.288468	1.0000	16.4075	10.1279	6.2796
Average	I-RRIP	0.290172	1.0068	14.9825	8.5170	6.4656

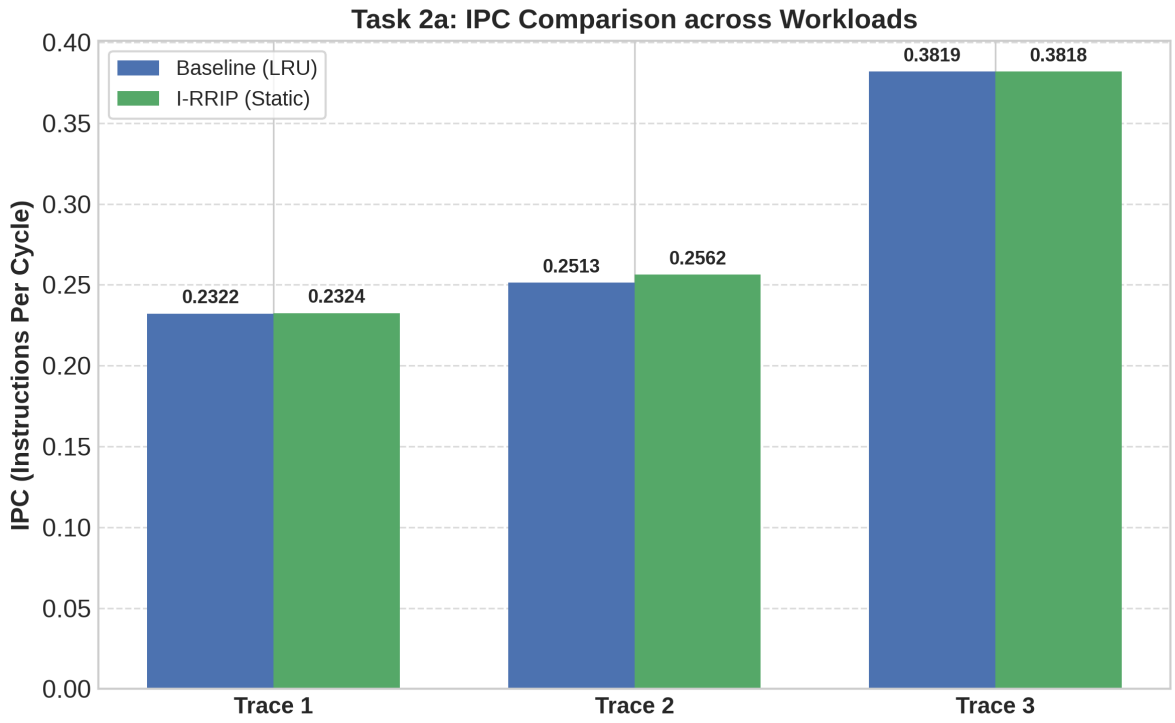
Table 2: Baseline vs static I-RRIP. Load MPKI = Instr MPKI + Data MPKI.

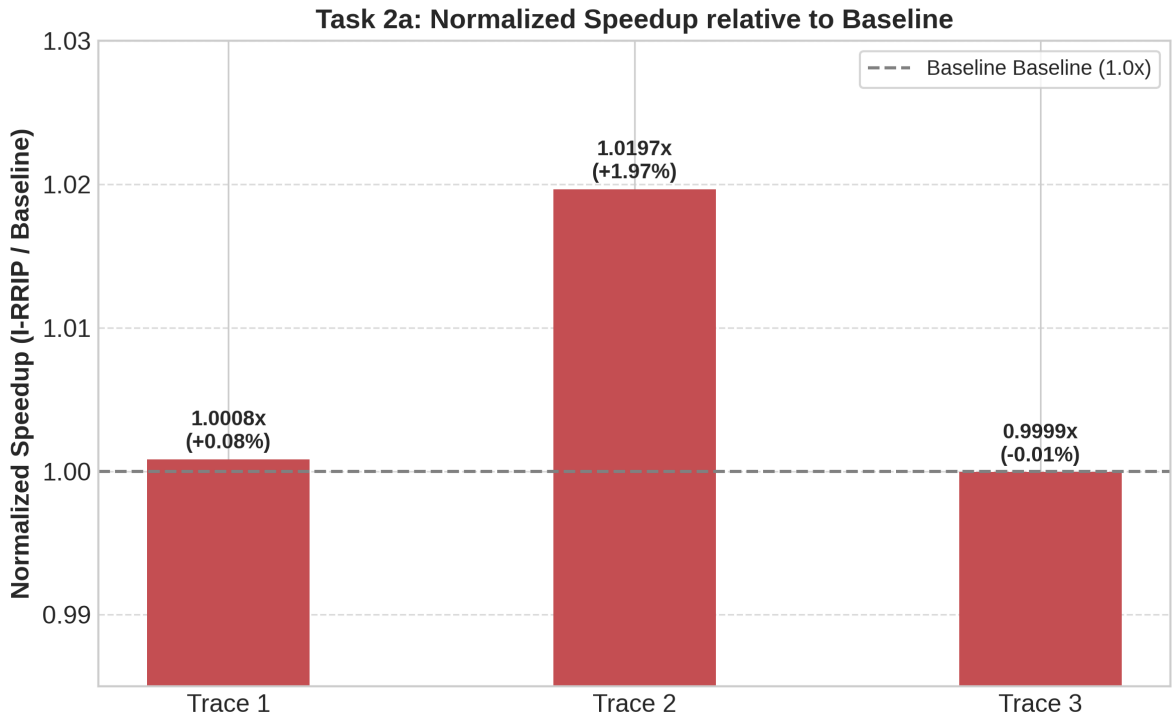
1.3 What changed in the instruction-bound trace (trace2)

Metric	Baseline	I-RRIP	Change
L2 instruction load MPKI	18.89	14.38	−23.9%
L2 data load MPKI	8.67	8.96	+3.3%
L2C total load MPKI	27.56	23.34	−15.3%
Instructions evicting instructions	295,827	191,336	−35.3%
Data evicting data	143,202	164,806	+15.1%

Table 3: Eviction-pair counters from the L2C statistics, trace2.

1.4 Plots





2 Task 2b: Dynamic I-RRIP

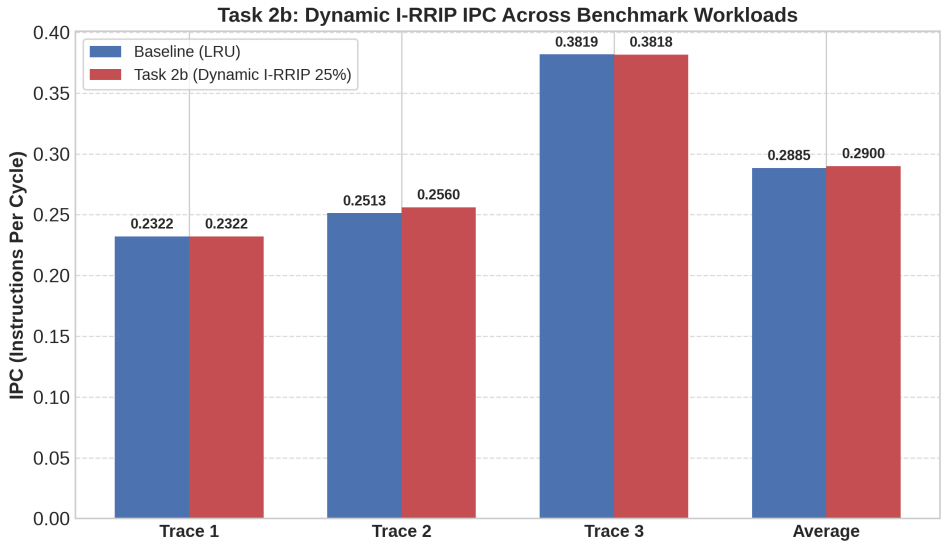
2.1 Results (threshold 25%, leader stride 32, epoch 4096)

Trace	Policy	IPC	Speedup	Load MPKI	Instr MPKI	Data MPKI
trace1	Dynamic I-RRIP	0.232161	0.9996	17.6584	10.1144	7.5441
trace2	Dynamic I-RRIP	0.256050	1.0189	23.3820	14.4892	8.8928
trace3	Dynamic I-RRIP	0.381800	0.9999	3.9560	1.0860	2.8700
Average	Dynamic I-RRIP	0.290004	1.0061	14.9988	8.5632	6.4356

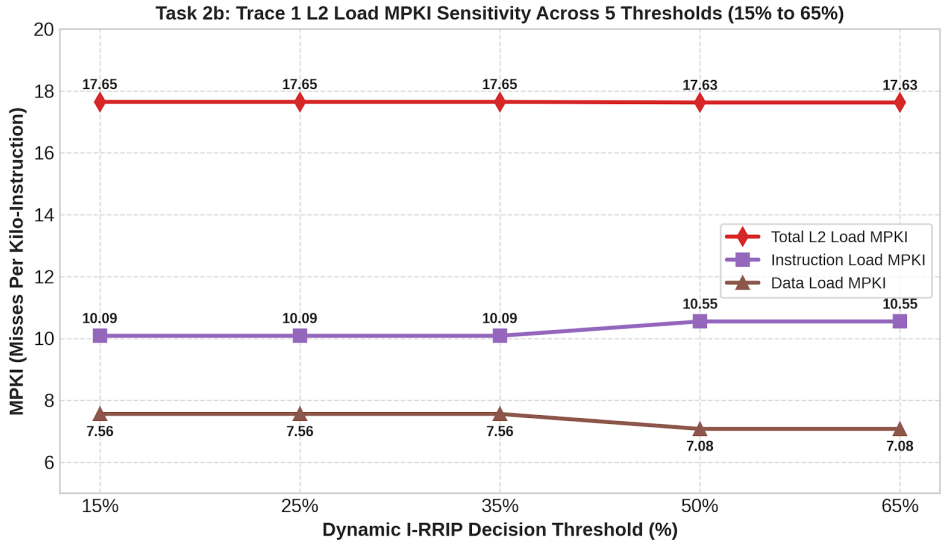
Table 4: Dynamic I-RRIP results.

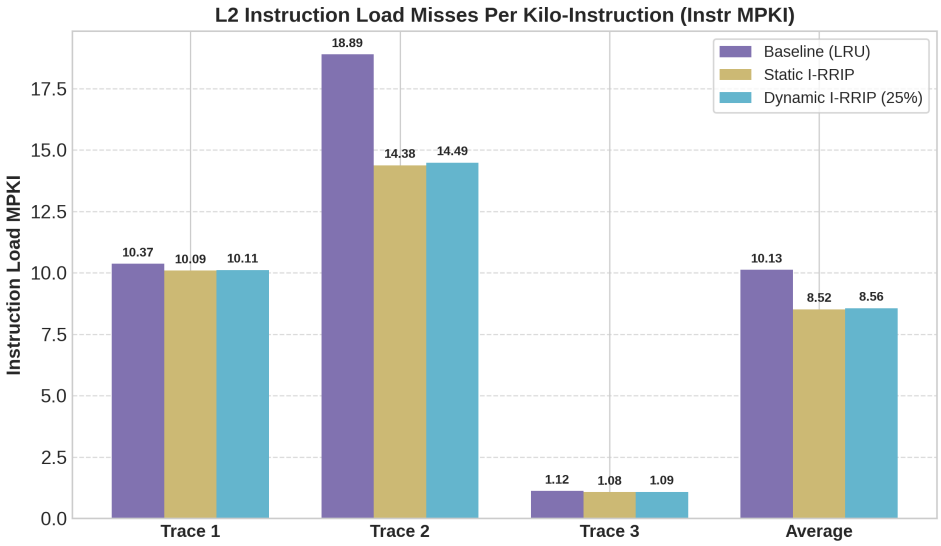
Policy	Avg IPC	Geomean speedup	Load MPKI	Instr MPKI	Data MPKI
Baseline LRU	0.288468	1.0000	16.4075	10.1279	6.2796
Static I-RRIP	0.290172	1.0068	14.9825	8.5170	6.4656
Dynamic I-RRIP	0.290004	1.0061	14.9988	8.5632	6.4356

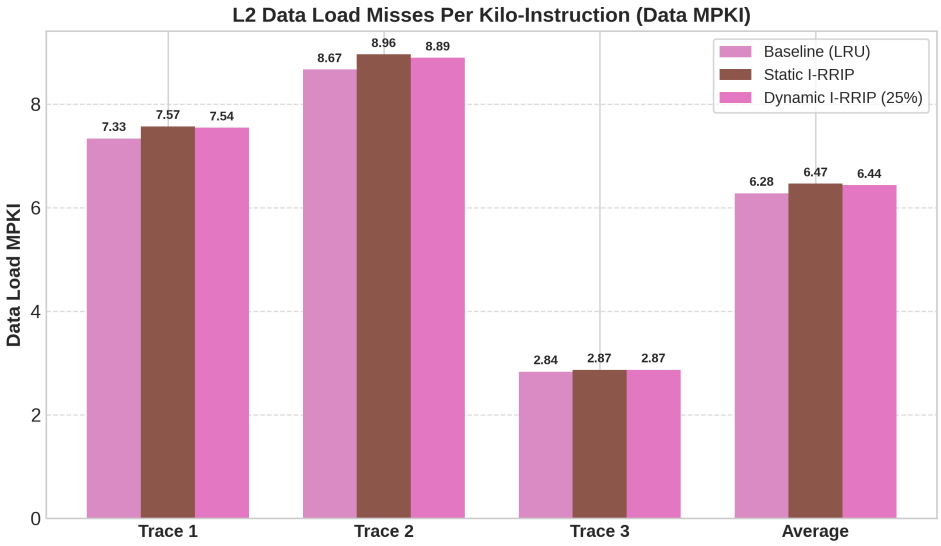
Table 5: Averages over the three traces.

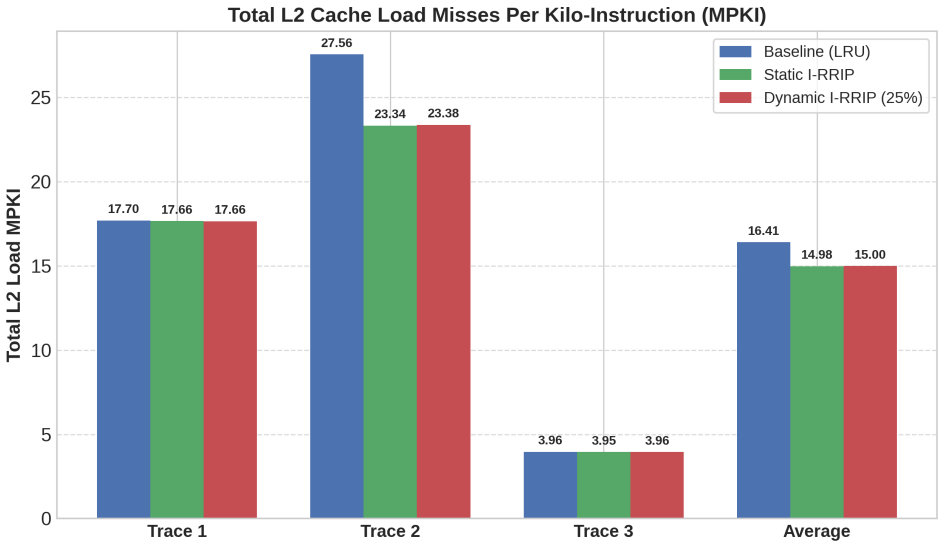

Task 2b: Dynamic I-RRIP IPC Across Benchmark Workloads
Baseline (LRU)
Task 2b (Dynamic I-RRIP 25%)
IPC (Instructions Per Cycle)
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.2322
0.2322
0.2513
0.2560
0.3819
0.3818
0.2885
0.2900
Trace 1
Trace 2
Trace 3
Average

2.2 Plots


Task 2b: Trace 1 L2 Load MPKI Sensitivity Across 5 Thresholds (15% to 65%)
MPKI (Misses Per Kilo-Instruction)
20
18
16
14
12
10
8
6
Total L2 Load MPKI
Instruction Load MPKI
Data Load MPKI
17.65
17.65
17.65
17.63
17.63
10.09
10.09
10.09
10.55
10.55
7.56
7.56
7.56
7.08
7.08
15%
25%
35%
50%
65%
Dynamic I-RRIP Decision Threshold (%)


L2 Instruction Load Misses Per Kilo-Instruction (Instr MPKI)
Instruction Load MPKI
Baseline (LRU)
Static I-RRIP
Dynamic I-RRIP (25%)
10.37
10.09
10.11
18.89
14.38
14.49
1.12
1.08
1.09
10.13
8.52
8.56
Trace 1
Trace 2
Trace 3
Average


L2 Data Load Misses Per Kilo-Instruction (Data MPKI)
Data Load MPKI
Baseline (LRU)
Static I-RRIP
Dynamic I-RRIP (25%)
7.33
7.57
7.54
8.67
8.96
8.89
2.84
2.87
2.87
6.28
6.47
6.44
Trace 1
Trace 2
Trace 3
Average


Total L2 Cache Load Misses Per Kilo-Instruction (MPKI)
Total L2 Load MPKI
Baseline (LRU)
Static I-RRIP
Dynamic I-RRIP (25%)
17.70
17.66
17.66
27.56
23.34
23.38
3.96
3.95
3.96
16.41
14.98
15.00
Trace 1
Trace 2
Trace 3
Average

2.3 Sensitivity study of the threshold

The sweep covers thresholds of 15%, 25%, 35%, 50% and 65% on all three traces (15 runs). Speedup is IPC over the LRU baseline; the last three rows of each table give the LRU baseline, Task 2a and the original Task 2b run for reference.

Threshold	IPC t1	IPC t2	IPC t3	Avg IPC	Sp. t1	Sp. t2	Sp. t3	Geomean
15%	0.232247	0.255522	0.382031	0.289933	1.0000	1.0168	1.0005	1.0057
25%	0.232247	0.255522	0.382031	0.289933	1.0000	1.0168	1.0005	1.0057
35%	0.232247	0.255201	0.382031	0.289826	1.0000	1.0155	1.0005	1.0053
50%	0.232206	0.254380	0.381726	0.289437	0.9998	1.0122	0.9997	1.0039
65%	0.232206	0.252063	0.381726	0.288665	0.9998	1.0030	0.9997	1.0008
LRU baseline	0.232246	0.251304	0.381855	0.288468	1.0000	1.0000	1.0000	1.0000
Task 2a (static)	0.232440	0.256243	0.381833	0.290172	1.0008	1.0197	0.9999	1.0068

Table 6: IPC and speedup versus threshold.

Threshold	Avg Load MPKI	Avg Instr MPKI	Avg Data MPKI
15%	15.0835	8.6618	6.4216
25%	15.0835	8.6618	6.4216
35%	15.1277	8.7302	6.3976
50%	15.3291	9.1320	6.1971
65%	15.7624	9.7288	6.0337
LRU baseline	16.4075	10.1279	6.2796
Task 2a (static)	14.9825	8.5170	6.4656

Table 7: L2C load MPKI versus threshold (averages over the three traces).

What the sweep shows.

- **Speedup falls as T rises.** Geomean speedup is 1.0057 at 15% and 25% (identical runs: every trace stays in I-RRIP mode in every epoch), 1.0053 at 35%, 1.0039 at 50% and 1.0008 at 65%. The loss is almost all trace2, whose speedup drops from 1.0168 to 1.0030.
- **The trade is visible in MPKI.** From 25% to 65%, average instruction MPKI rises from 8.66 to 9.73 and data MPKI falls from 6.42 to 6.03. A higher threshold switches protection off, giving back instruction hits for data hits.
- **Mode counts show where each trace flips.** Trace3 is on at 25%, 5 of 6 epochs at 35% and off at 50%. Trace1 is on at 35% and off at 50%. Trace2 sits near the boundary at 50%–65% (14 of 19 epochs on at 50%, 3 of 19 at 65%). So the leader-measured instruction share is roughly 25–35% for trace3, 35–50% for trace1 and 50–65% for trace2. This corrects my earlier estimate for trace3 (22%).
- **No threshold beats static I-RRIP** (1.0068). The best dynamic setting reaches 1.0057. On trace3 the dynamic run is slightly ahead of static (1.0005 against 0.9999), but 0.05% is within the noise of these runs.
- **Baseline mismatch is small.** At 50% and above, trace1 and trace3 run in SRRIP mode for almost the whole run. Their IPC (0.232206 and 0.381726) is only 0.02% and 0.03% below LRU, so the SRRIP-versus-LRU difference barely affects the other results.

2.4 Rationale for the leader sets and decision logic

The design (`dyn_irrip.l2c_repl`) follows the assignment's set-dueling scheme. Leader sets always run plain RRIP and collect statistics; the follower sets run I-RRIP or RRIP depending on a formula over those statistics. The leaders do not compare two policies' hit counts: they measure **how much of the demand traffic is instruction traffic**, and the formula turns that share into a mode.

Leader sets: every 32nd set. With 1024 L2C sets that is 32 leaders, about 3.1% of the cache. The other 992 follower sets run I-RRIP when the mode is on and SRRIP when it is off.

- **Why leaders always run SRRIP.** The statistic they gather should not depend on the mode being decided. If leaders ran I-RRIP while on, the policy would change the traffic it is measuring and the decision could chase itself.
- **Why 32 and not fewer.** Few leaders give a noisy sample. 32 sets spread evenly over the index space cover many address regions.
- **Why not many more.** Leaders are forced into SRRIP even when I-RRIP is winning, so each extra leader costs a little of the gain. At 3.1% at most about 3% of sets are forced off the better policy.
- **Why a regular stride.** `set % 32 == 0` is a single compare, with no tag or table.

Decision logic. Four counters are kept: instruction and total demand accesses, and instruction and total demand misses. Writebacks are ignored because they are not demand traffic. At the end of each epoch:

$$(i_{acc} + i_{miss}) \times 100 \;>\; T \times (t_{acc} + t_{miss}) \;\Rightarrow\; \text{I-RRIP on, else off.}$$

Counting misses in addition to accesses weights instruction misses double, matching their cost, since an instruction miss stalls the front end. Only an integer multiply and compare are needed.

Epoch: 4096 leader events. Long enough to gather thousands of samples per decision, so the mode does not flip on noise; short enough to follow phase changes. After each decision the counters are *halved* instead of cleared, so the previous phase fades out gradually, giving built-in hysteresis. Before the first decision the mode is I-RRIP (`INIT_IRRIP_MODE = 1`), the safe default because Task 2a rarely hurts.

What the sweep measured for the three traces. At $T = 25\%$ every trace stayed in I-RRIP mode for every epoch (11 of 11, 19 of 19, 6 of 6), so Task 2b at 25% behaves like Task 2a. That is why trace2 keeps nearly all of its static gain (+1.68% against +1.97%) and why traces 1 and 3 are flat. The epoch counts at higher thresholds locate each trace's instruction share between about 25% and 65%.

2.5 Does dynamic beat static?

Not on these three traces.

- Dynamic is slightly behind static on every trace (1.0189 vs 1.0197 on trace2; 0.9996 vs 1.0008 on trace1); the geomean gap is 0.07 percentage points.
- It reduces the cost: the trace2 data-MPKI penalty shrinks from +3.3% to +2.6%, and trace1 from +3.2% to +2.9%.

- It gives back some instruction benefit (trace2 instruction MPKI 14.49 vs 14.38), most likely because of the 32 SRRIP leader sets and any epochs spent in SRRIP mode.

Dynamic is the right design for data-dominated workloads where static I-RRIP would cost real IPC; none of these traces is like that (worst static result: -0.006%). Its case here rests on robustness, not measured gain. On these traces static wins by a very small margin. The IPC differences on traces 1 and 3 ($\sim$0.0001–0.0002) are small; only trace2 shows a clearly real effect.

Baseline mismatch. Every output file reports an LRU baseline at L2C, while the SRRIP mode in `dyn_irrip.l2c_repl` is plain SRRIP. In Task 2b the leaders, and followers during any SRRIP epoch, therefore differ from the baseline by the SRRIP-versus-LRU effect as well as by I-RRIP. For a clean attribution, add an SRRIP-only run on the same three traces.

CS683 Programming Assignment II
Task 3: Integrating SPP with Prefetch-Aware Dynamic I-RRIP

1. Objective and implementation

Task 3 modifies the L2 replacement policy to recognize lines generated by L1D SPP, while retaining Dynamic I-RRIP's instruction-awareness. Its performance objective is to improve instructions per cycle (IPC) relative to the standalone components.

Prefetch-aware policy. Re-reference prediction values (RRPVs) range from 0 (strongly retained) to 3 (most eligible for eviction). In I-RRIP mode, instruction-demand fills enter at RRPV 1 and demand-data fills at RRPV 2. Instruction prefetches enter at RRPV 2; data prefetches enter at RRPV 3. Prefetch hits do not cause promotion, because a prefetch request does not itself establish demand reuse. Leader sets continue to use ordinary SRRIP, while follower sets dynamically select instruction-aware I-RRIP or SRRIP.

Dynamic selection. Every 32nd L2 set is a leader set. Demand-only leader statistics exclude prefetch, writeback, and translation requests. Every 4096 eligible accesses, the policy evaluates

$$P_I = \frac{\text{instruction accesses} + \text{instruction misses}}{\text{total accesses} + \text{total misses}}.$$

If $P_I > 0.25$, followers select I-RRIP; otherwise they select SRRIP. All observed evaluation epochs on the three measured traces selected I-RRIP.

2. IPC results

Benchmark	SPP only*	SPP disabled	Combined Task 3	Gain vs SPP
Trace 1	0.234213	0.232247	0.234111	−0.044%
Trace 2	0.253038	0.255522	0.257694	+1.840%
Trace 3	0.389045	0.382031	0.389107	+0.016%
Geometric-mean gain	–	–	–	+0.600%

*Standalone SPP IPCs use the Task 1 baseline measurements; the "SPP disabled" column uses the same L2 replacement policy without L1D SPP.

Interpretation. The combined configuration improves IPC relative to the SPP-disabled L2 policy on all three traces. Against standalone SPP, IPC improves on Traces 2 and 3 and is 0.044% lower on Trace 1. Its geometric-mean IPC gain relative to standalone SPP is approximately 0.60%.

3. Dynamic and prefetch-policy diagnostics

Statistic	Trace 1	Trace 2	Trace 3
Evaluation epochs	11	19	5
Epochs selecting I-RRIP	11	19	5
Mode switches	0	0	0
L2 prefetch fills (policy counter)	90,711	52,359	72,975
Prefetch hits not promoted	26,976	7,179	4,623

Nonzero fill and unpromoted-hit counters confirm execution of the prefetch-aware paths. Zero observed mode switches mean the 25% threshold was not crossed during these measurement windows, not that switching would be unnecessary on other workloads.

4. Performance visualization

Figure 1 compares absolute IPC across three configurations. Figure 2 shows load miss intensity at L1D and L2C for the combined configuration; these metrics characterize different levels of the cache hierarchy and should not be directly summed.

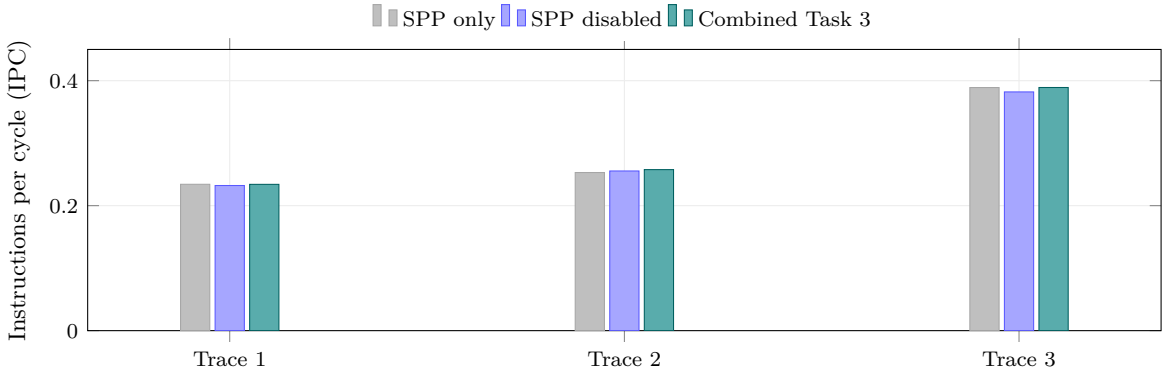


Figure 1: IPC across the three benchmark traces at the prescribed 25M/25M instruction settings.

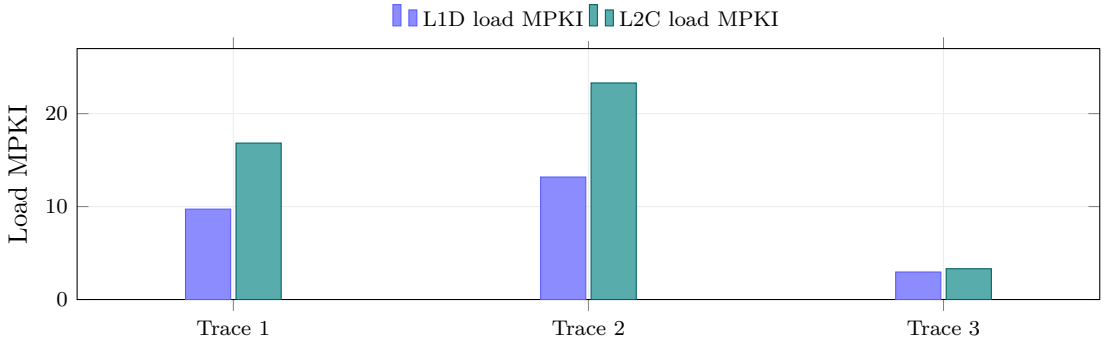


Figure 2: Load misses per thousand instructions under the combined policy.

5. Cache and prefetch metrics

MPKI is the number of cache misses per thousand measured instructions. The following measurements are from the three completed Task 3 simulation logs.

Metric	Trace 1	Trace 2	Trace 3
L1D load misses	243,122	329,502	74,004
L1D load MPKI	9.72488	13.18010	2.96016
L2C load MPKI	16.83290	23.30040	3.31368
L2C instruction load MPKI	10.08180	14.79200	1.07980
L2C data load MPKI	6.75108	8.50840	2.23388
L1D SPP accuracy (%)	56.2569	48.8721	45.6523
L1D useful prefetches	32,094	11,353	12,002
L1D prefetches issued to lower level	57,049	23,230	26,290

Instruction and data L2C load MPKIs sum to total L2C load MPKI, subject to rounding.

L1D prefetch coverage. We use

$$\text{Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}_{\text{combined}}}{\text{L1D load misses}_{\text{baseline}}} \times 100\%.$$

The baseline measurements provide rounded MPKIs (10.678, 13.538, and 3.403). Consequently, the following percentages are *approximations* computed using MPKI rather than exact baseline miss counts.

Trace	Baseline L1D MPKI*	Combined L1D MPKI	Approx. coverage
Trace 1	10.678	9.72488	8.93%
Trace 2	13.538	13.18010	2.64%
Trace 3	3.403	2.96016	13.01%

*Baseline MPKIs are drawn from the Task 1 baseline measurements under the same specified instruction counts. Exact coverage requires the baseline's unrounded L1D load miss counts.

Accuracy definition. L1D SPP accuracy is the simulator's useful load prefetches divided by prefetch requests issued to the lower level. This differs from the L2 prefetch accuracy metric, which measures demand usefulness at another cache level.

6. Build and reproducibility

The implementation consists of `prefetcher/spp.l1d_pref`, `replacement/dyn_irrip.l2c_repl`, and modified `inc/cache.h`, `src/cache.cc`, and `src/main.cc`. Simulator modifications propagate instruction-line metadata into the replacement policy and support extensionless raw traces. These common simulator modifications are required to reproduce the combined configuration.

Build: `./build_champsim.sh spp dyn_irrip task3`

Run: `-warmup_instructions 25000000 -simulation_instructions 25000000 -traces traces/TraceN`

Validation status. Compilation and three full-length trace runs completed with exit code 0. Coverage is approximate because only rounded baseline load MPKIs are available. Trace inputs are not embedded in the source distribution.